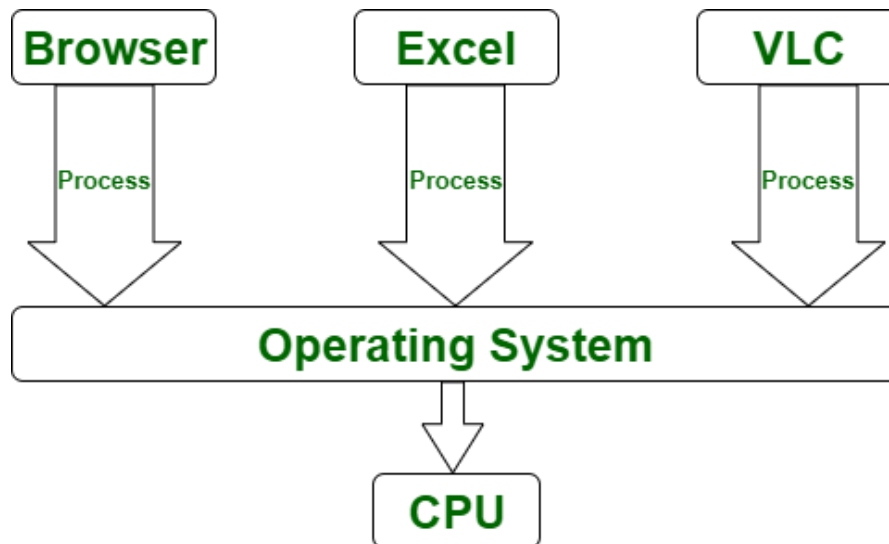


Multithreading

What is Multitasking ?

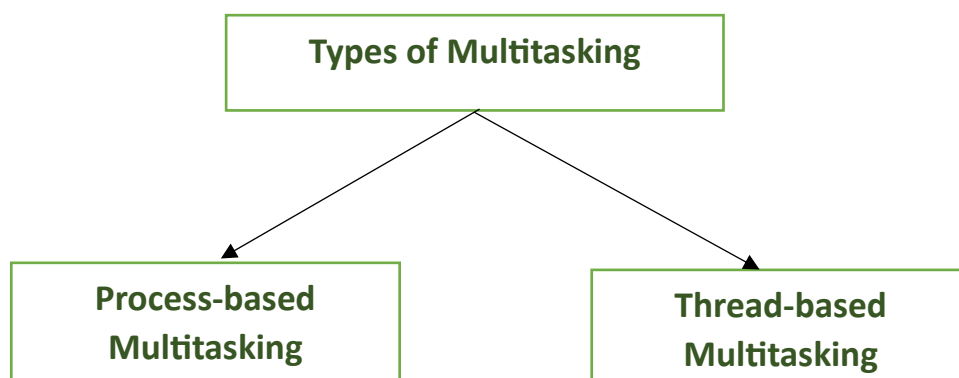
- Multitasking means the CPU executes multiple tasks at a time.
- It allows the CPU to switch quickly between tasks.
- Users can work with multiple programs simultaneously.
- Unlike multithreading, each process has separate memory and resources.



Types of Multitasking :-

There are mainly two types of multitasking:

1. Process-based (Multiprocessing)
2. Thread-based (Multithreading)



1. Process-based Multitasking :-

- In this type, multiple programs (processes) run at the same time.
- Example: Running a web browser, music player, and word processor simultaneously.

2. Thread-based Multitasking :-

- In this type, multiple threads of a single process execute concurrently.
- Example: A web browser loading multiple tabs at once.

Advantages of Multitasking :-

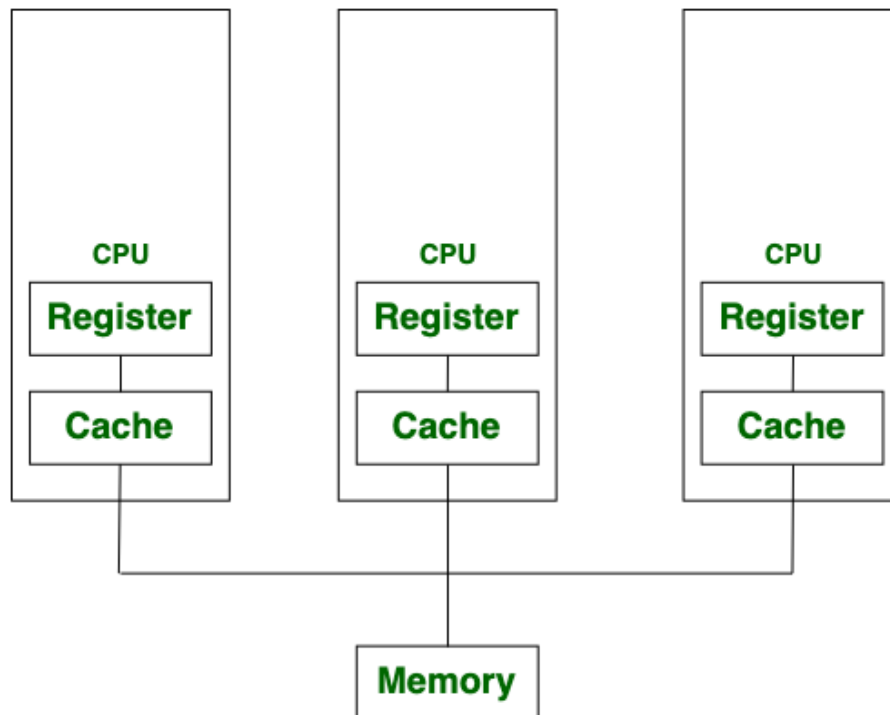
- 1. Increased Productivity:** Multiple processes run at once for better work output.
- 2. Resource Utilization:** CPU and system resources are used efficiently.
- 3. Enhanced Efficiency:** Runs several tasks, making full use of the CPU.

Disadvantages of Multitasking :-

- 1. Overhead:** Task switching causes extra processing load.
- 2. Slower Performance:** Slower than multithreading since each process has its own memory.

What is Multiprocessing ?

- Multiprocessing is a system that has two or more processors.
- Each processor works independently.
- It allows multiple processes to run at the same time.
- It increases the speed and performance of the system.



Advantages of Multiprocessing :-

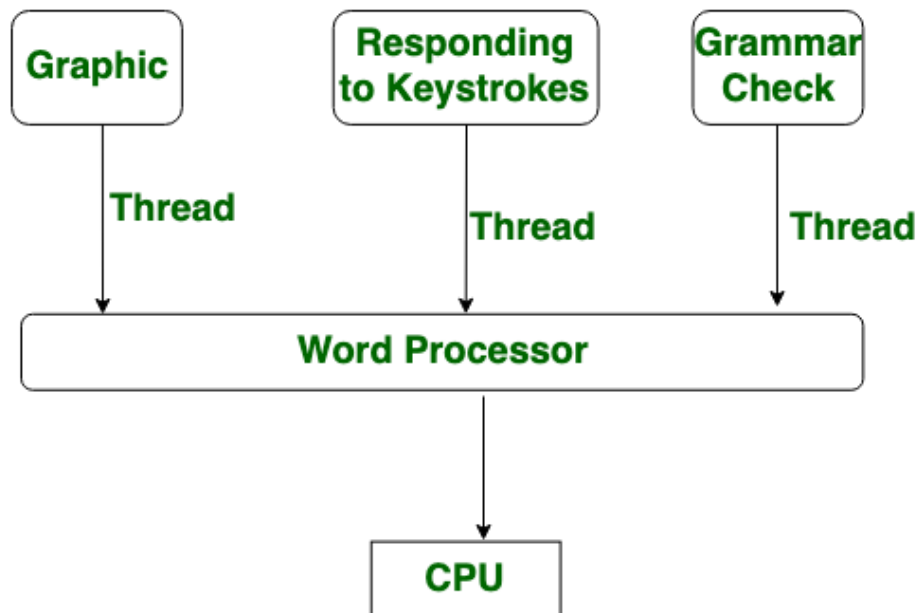
1. Increases computing power using multiple processors.
2. Best for tasks needing high computation.

Disadvantages of Multiprocessing :-

1. Process creation takes more time.
2. Uses more memory due to separate address spaces.

What is Multithreading ?

- Multithreading allows a process to create multiple threads.
- It increases the computing power of the system.
- The CPU executes many threads of a process at the same time.
- Thread creation is less costly than process creation.



Advantages of Multithreading :-

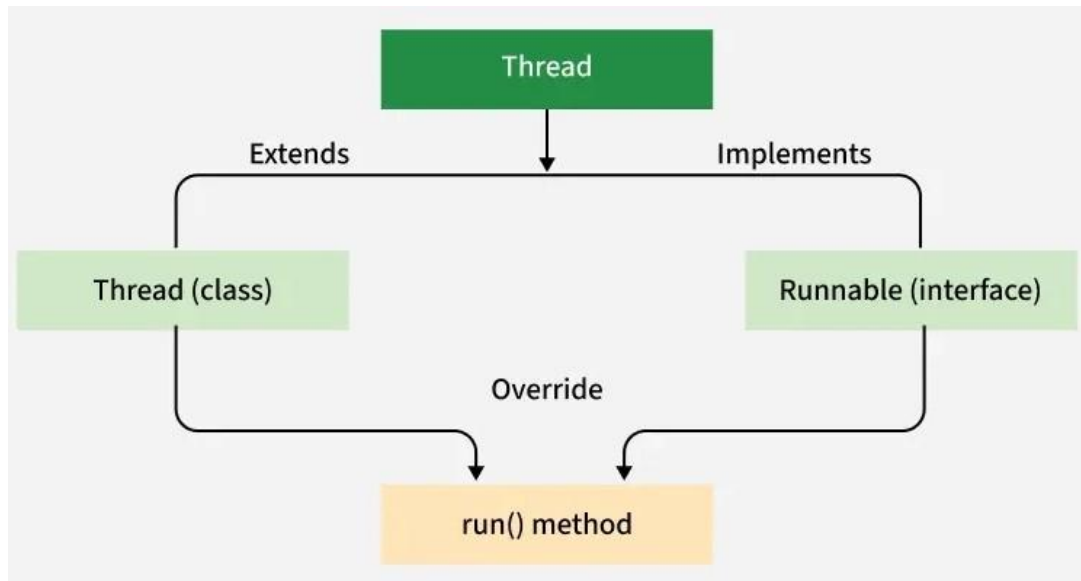
1. **Faster Execution:** Threads share memory and resources, making execution quicker.
2. **Efficient Resource Use:** Context switching is faster and uses fewer resources.
3. **Better Performance:** Threads run code in parallel, improving speed and responsiveness.

Disadvantages of Multithreading :-

1. **Complexity:** Managing thread synchronization and communication is difficult.
2. **No Memory Protection:** Threads share memory, increasing the risk of data corruption.

How to achieve Multithreading ?

- Multithreading is achieved by creating and running multiple threads.
- There are two main ways to create threads in Java:
 1. By extending the Thread class
 2. By implementing the Runnable interface
- Each thread runs its code inside the run() method.
- To start a thread, call the start() method, not run().
- The JVM and CPU scheduler manage thread execution automatically.



1) Thread Class :-

- The Thread class is found in the java.lang package.
- It is used to create and manage threads in Java.
- A thread represents a separate path of execution in a program.

Steps to Achieve Multithreading using Thread Class -

1. Create a class that extends the Thread class.
2. Override the run() method — write the code you want the thread to execute.
3. Create an object of your thread class.
4. Call the start() method to begin thread execution (this internally calls run()).
5. The JVM schedules the threads to run concurrently.

Example 1 –

```
package multithreading;
//1.extends the Thread class
public class multi_threading extends Thread{
//2.override run method or write down the run method for particular subtask
    public void run()
    {
        System.out.println("Thread 1 is Running");
    }
    public static void main(String[] args) {
        //3.create the object of class
        multi_threading t1=new multi_threading();
        //4.executes the thread
        t1.start();
    }
}
```

Output –

Thread 1 is Running

Example 2 –

```
package multithreading;

public class youtube extends Thread{
    void play_video()
    {
        System.out.println("Video is Playing");
    }
    void play_audio()
    {
        System.out.println("Audio is Playing");
    }
    void play_caption()
    {
        System.out.println("Caption is Displaying");
    }
    public void run()
    {
        play_video();
        play_audio();
        play_caption();
    }
    public static void main(String[] args) {
        youtube y=new youtube();
        y.start();
    }
}
```

Output –

Video is Playing

Audio is Playing

Caption is Displaying

Methods of Thread Class :-

1. **start()** – Starts the thread and calls the `run()` method internally.
2. **run()** – Contains the code to be executed by the thread.
3. **sleep(long ms)** – Makes the thread pause for a given time in milliseconds.
4. **join()** – Waits for another thread to finish before continuing.
5. **getName()** – Returns the name of the thread.
6. **setName(String name)** – Sets a new name for the thread.
7. **getPriority()** – Returns the priority of the thread.
8. **setPriority(int p)** – Sets the priority of the thread (1 to 10).
9. **isAlive()** – Checks if the thread is still running.
10. **currentThread()** – Returns a reference to the currently executing thread.
11. **yield()** – Pauses the current thread to let other threads execute.
12. **interrupt()** – Interrupts a sleeping or waiting thread.
13. **isInterrupted()** – Checks if the thread has been interrupted.

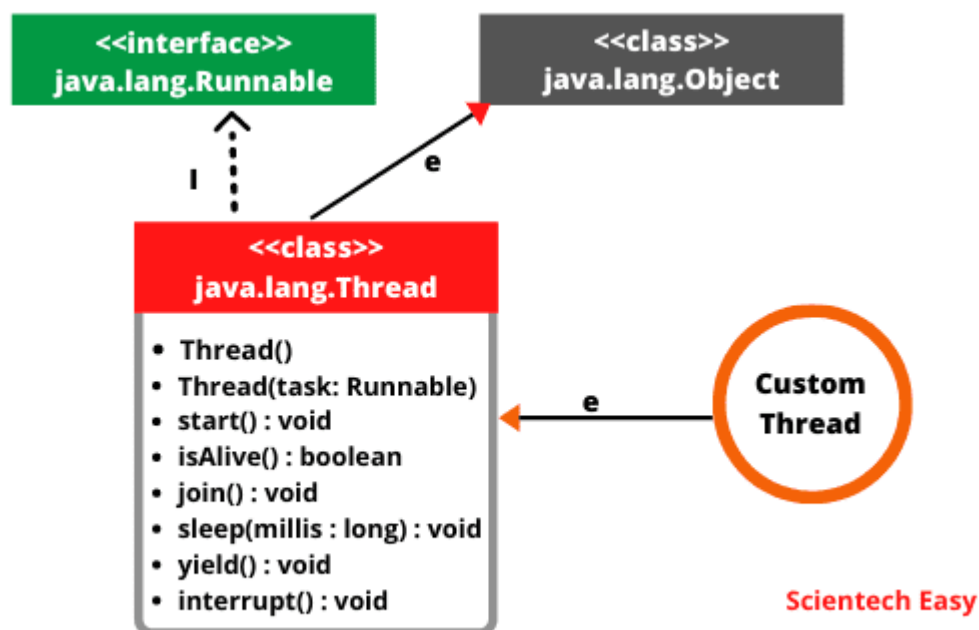


Fig: Thread class contains methods for controlling threads

Example –

```
package multithreading;

public class thread_methods {

    public static void main(String[] args) {
        Thread t1=new Thread();
        t1.setName("Audio_Thread");
        System.out.println("Name: "+t1.getName());

        t1.setPriority(3);
        System.out.println("Priority: "+t1.getPriority());

        System.out.println("ID: "+t1.getId());
        System.out.println("Min: "+Thread.MIN_PRIORITY);
        System.out.println("Norm: "+Thread.NORM_PRIORITY);
        System.out.println("Max: "+Thread.MAX_PRIORITY);
        System.out.println(t1.isAlive());
        System.out.println(t1.isVirtual());
        System.out.println(t1.isDaemon());
    }
}
```

Output –

```
Name: Audio_Thread
Priority: 3
ID: 21
Min: 1
Norm: 5
Max: 10
false
false
false
```

Difference Between Multitasking Multiprocessing & Multithreading :-

Aspect	Multitasking	Multiprocessing	Multithreading
1. Definition	Running multiple tasks at a time.	Using two or more processors simultaneously.	Running multiple threads within one process.
2. Processor Usage	Single CPU switches between tasks.	Multiple CPUs work in parallel.	Single CPU handles multiple threads.
3. Execution Type	Concurrent execution of tasks.	True parallel execution.	Concurrent execution within one process.
4. Memory Usage	Each task has its own memory.	Each process has separate memory.	Threads share same memory.
5. Communication	Difficult between tasks.	Difficult between processes.	Easy between threads.
6. Speed	Moderate speed.	Fastest due to parallelism.	Faster than multitasking.
7. Example	Running Paint and Browser together.	Using multi-core CPU for programs.	A program downloading file and updating UI together.
8. Overhead	High context switching overhead.	High process creation overhead.	Low overhead due to shared resources.

Difference Between Process & Thread :-

Aspect	Process	Thread
1. Definition	A program in execution	A lightweight unit of a process
2. Memory	Has its own memory space	Shares memory of the process
3. Creation	Slower	Faster
4. Communication	Inter-process communication (IPC) needed	Easy within same process
5. Overhead	High	Low
6. Execution	Independent	Dependent on process
7. Example	Running Chrome	Tabs in Chrome